

Multithreading

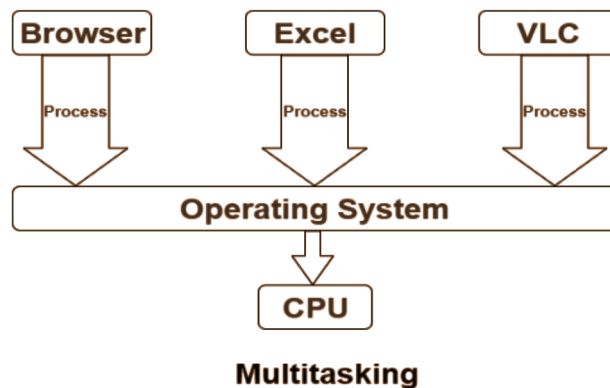
1. Multi Tasking

a. Multi Thread

b. Multi Processing

Multitasking

- CPU executes multiple jobs by switching among them.
- typically using a small-time quantum, and these switches occur so frequently that the users can interact with each program while it is running.



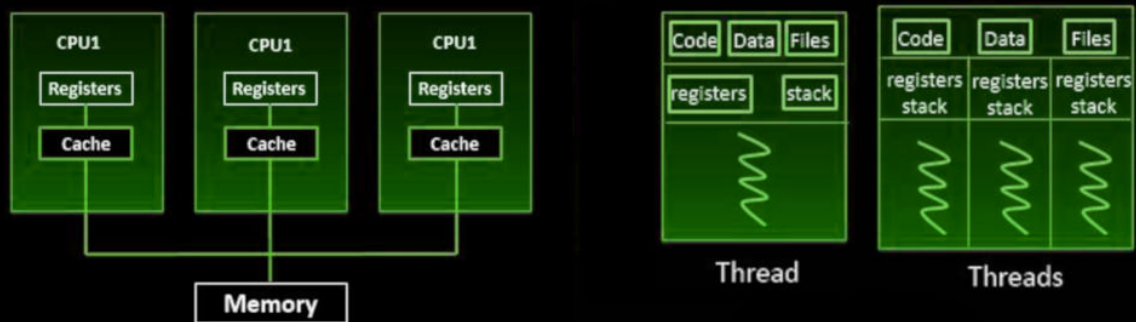
Multi Processing:

- Process is a program.
- Each process is treated as a separate execution unit and is assigned to a different CPU.
 - Running MS Word C#
 - Running PPT
 - Running Zoom
 - Running WhatsApp
- System Level concept. CPU level, Raspberry Pi 1 Single CPU
- They will have **different memories (written in different languages)**.
- They can share data, but it is a very complex process.
- Context switch or control is difficult. (P1 will ask P2, P3 for permission)

Multi Threading:

- Thread is part of Program
 - **Thread is a sub process.**
 - E.g. Zoom,
 - SP1, SP2, SP3 three sub processes each are Thread
 - SP1 - Audio transfer
 - SP2 - Video Recordings
 - SP3 - Chat
 - Application level
 - They can share the memories
 - They can do it easily in threads.
 - Switching to control in Threads is easy. (T1 -> T2)
- Bank Application allows multiple users to login, bank balance, withdraw amount.
-

Multiprocessing vs Multithreading



▶ Multitasking vs Multithreading vs Multiprocessing

```
public class Task02 {
```

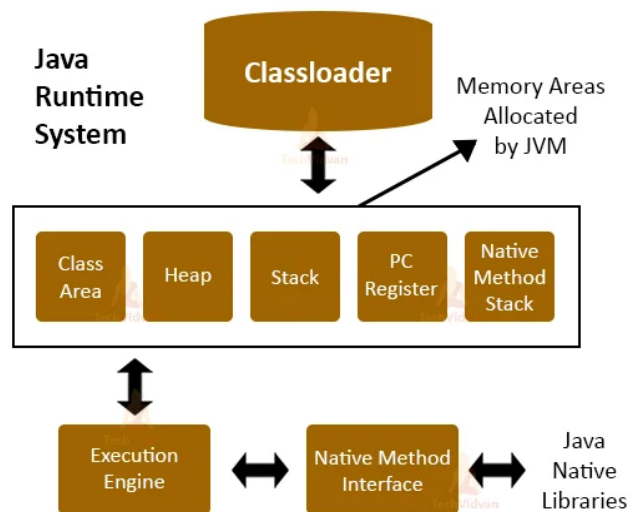
```

    public static void main(String[] args) throws
InterruptedException {

    Thread t = Thread.currentThread();
    System.out.println(t);
    for (int i = 0; i < 10; i++) {
        System.out.println(i+ " - "+t.getName());
        System.out.println(i+ " - "+t.getPriority());
        Thread.sleep(5000);
    }
}
}

```

JVM Architecture



Flow of Java program

1. **Java Task02 123**
2. The JVM asks the OS for the memory.
3. If the OS allocates the memory -> JVM adds Stack, Heaps.
4. JVM creates 2 thread groups, Main and System (2-3 threads). (GC, finalizer)
5. JVM adds the main thread to the main group.
6. JVM started the main thread. The main thread will do the program task.
7. Check whether **Task02 .class** is available or not.
8. ByteCode verification will be done.
9. Collect parameters (123).

10. Verify that the main method is present.
11. Create a String array and store command-line args.
12. Invokes `ClassName.main()` (`Task02.main()`) by JVM
13. The main thread will execute your program.
- 14.
15. JVM will SJVM will clear garbagehut down.

Main Thread - Main Method

There are two ways you can create the Threads in Java.

1. Extend Thread
2. Runnable

```
package thetestingacademy.multithreading;

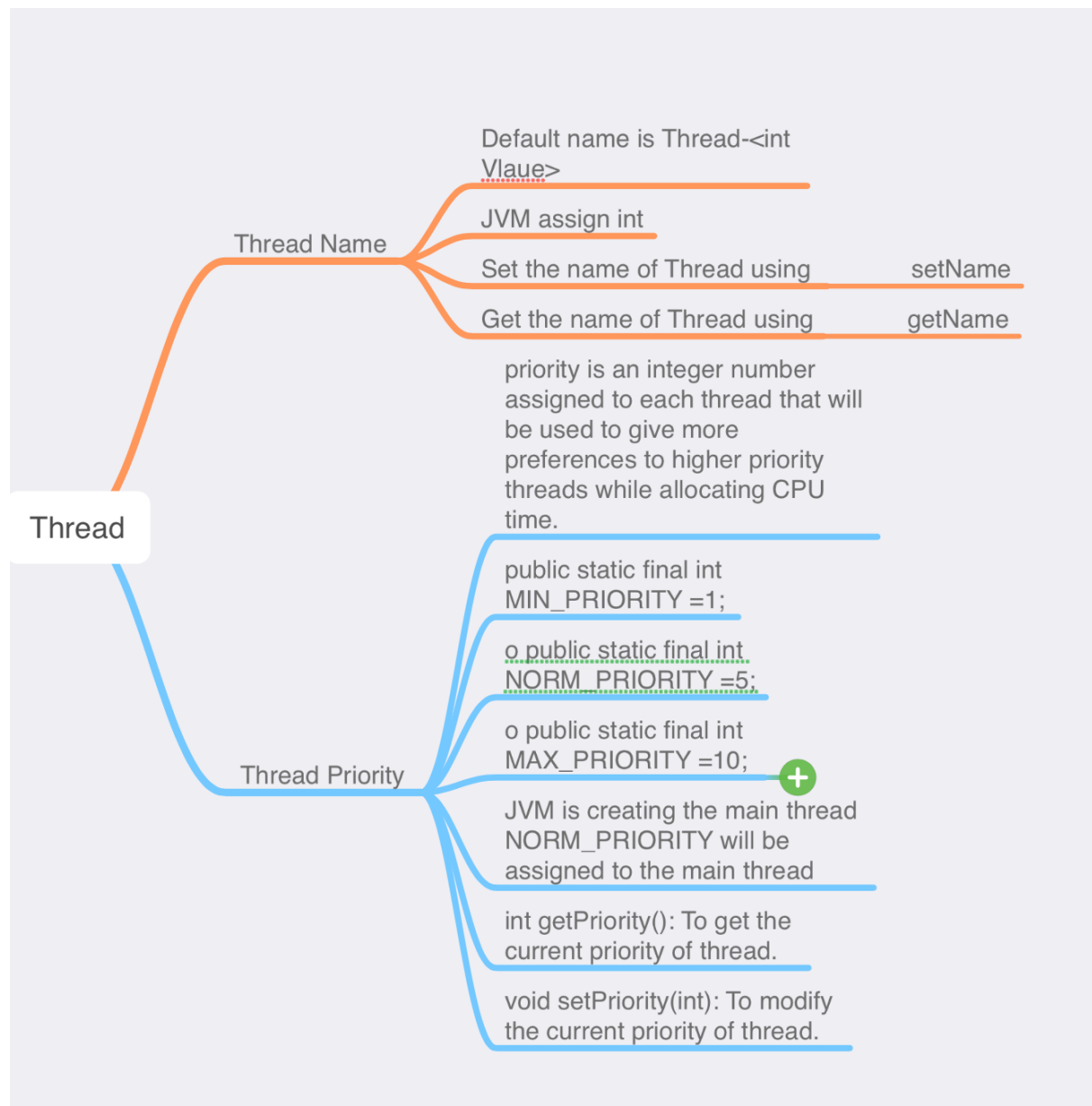
public class Task03 {
    public static void main(String[] args) {

        Worker worker = new Worker();
        worker.start();
        // Where is the Start ?
        Worker worker2 = new Worker();
        worker2.start();

    }
}

class Worker extends Thread{

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            try {
                System.out.println(i + " --> ");
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

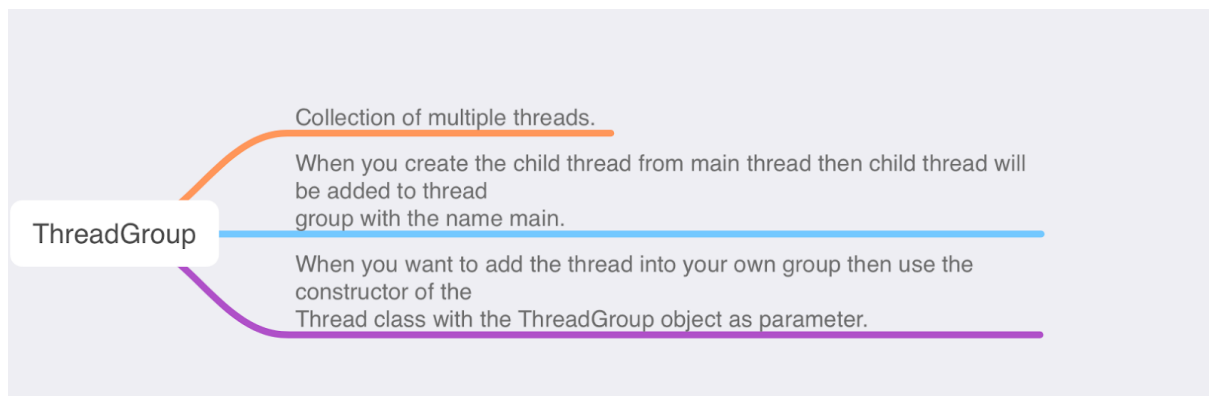


Why don't we call run() method directly, why call start() method?

- We can call run() method if we want but then it would behave just like a normal method and we would not be able to take the advantage of multithreading.
- When the run method gets called through start() method then a new separate thread is being allocated to the execution of run method, so if more than one thread calls start() method that means their run method is being executed by separate threads (these threads run simultaneously).
- public Thread():
Creates a new Thread object. When started, then run() will be invoked from the current object.
- public Thread(String);

Creates a new Thread object by specifying the name.

- `public Thread(ThreadGroup, String);`
Creates a new Thread object by specifying the ThreadGroup name.
- `public Thread(Runnable);`
Creates a new Thread object. When started then `run()` will be invoked from specified Runnable object.
- `public Thread (Runnable, String);`
Creates a new Thread object by specifying the Runnable object and thread name.
- `public Thread(ThreadGroup, Runnable);`
Creates a new Thread object by specifying the threadgroup Runnable object.
-
- `public Thread(ThreadGroup, Runnable, String);`
Creates a new Thread object by specifying the ThreadGroup, Runnable object and thread name.



Methods from ThreadGroup Class

- `public final String getName()` **Returns the name of this thread group.**
- `public final ThreadGroup getParent()` **Returns the parent of this thread group**
- `public final boolean parentOf(ThreadGroup)`
Tests if this thread group is either the thread group argument or parent of that thread group
- `public final int getMaxPriority()` **Returns the maximum priority of this thread group.**
- `public final void setMaxPriority(int)` **Sets the maximum priority of the group.**
- `public final boolean isDaemon()` **Tests if this thread group is a daemon thread group.**
- `public final void setDaemon(boolean)` Changes the daemon status of this thread group.
- `public synchronized boolean isDestroyed()`

Tests if this thread group has been destroyed.

- public int activeCount() Returns an estimate of the number of active threads in this thread group and its subgroups.
- public final void stop() Stops this thread group.

<u>By extending Thread class</u>	<u>By implementing Runnable interface</u>
♦ Define your own thread class by extending java.lang.Thread class. ♦ Override the run() method with following signature. public void run() ♦ Write the logic required for your thread inside the run() method. ♦ Create the object of Thread sub class written by you ♦ Call the start() method using your own thread object.	♦ Define your own thread class by implementing java.lang.Runnable interface. ♦ Override the run() method with following signature. public void run() ♦ Write the logic required for your thread inside the run() method. ♦ Create the object of java.lang.Thread by passing Runnable object as parameter to constructor. ♦ Call the start() method using thread object.

Constants from Thread Class

- static final int MIN_PRIORITY Constant represents **minimum priority of a thread**
- static final int NORM_PRIORITY Constant represents **normal priority of a thread**
- static final int MAX_PRIORITY Constant represents **maximum priority of a thread**

Methods from Thread Class

Methods from Thread Class	
static native Thread currentThread()	Returns a reference to the currently executing thread object.
synchronized void start()	Causes this thread to begin execution; the Java Virtual Machine calls the run method of this thread.
void run()	Inherited from Runnable Interface
long getId()	Returns the identifier of this Thread.
final native boolean isAlive()	Tests if this thread is alive.
final void stop()	Stops this thread execution.
final void setPriority(int)	Changes the priority of this thread.
final int getPriority()	Returns this thread's priority.
final synchronized void setName(String)	Changes the name of this thread.
final String getName()	Returns this thread's name.
final ThreadGroup getThreadGroup()	Returns the thread group to which this thread belongs.
static int activeCount()	Returns an estimate of the number of active threads in the current thread's thread group and its subgroups.
boolean isInterrupted()	Tests whether the current thread has been interrupted.
void interrupt()	Interrupts this thread.
final void setDaemon(boolean)	Marks this thread as either a daemon thread or a user thread.
final boolean isDaemon()	Tests if this thread is a daemon thread.

Thread.State getState()	Returns the state of this thread.
static native void yield()	A hint to the scheduler that the current thread is willing to leave its current use of a processor.
static native void sleep(long)	Causes the currently executing thread to sleep (temporarily pause the execution) for the specified number of milliseconds.
static void sleep(long, int)	Causes the currently executing thread to sleep (temporarily pause the execution) for the specified number of milliseconds plus the specified number of nanoseconds
final synchronized void join(long)	Waits at most millis milliseconds for this thread to die.
final synchronized void join(long, int)	Waits at most millis milliseconds plus nanos nanoseconds for this thread to die.
final void join()	Waits for this thread to die.

Concurrency vs Parallelism

Concurrency refers to the ability of a system to handle multiple tasks or threads at the same time, allowing them to be executed independently and asynchronously.

Concurrency enables a system to perform multiple tasks simultaneously, but it does not necessarily mean that the tasks are executed simultaneously.

For example, **a web server can handle multiple client requests concurrently**, but each request is handled by a single thread and the server may not be able to process all the requests at the same time.

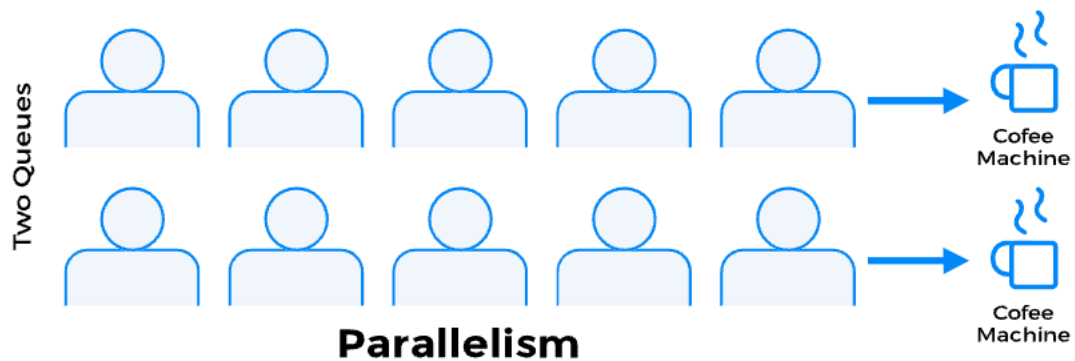
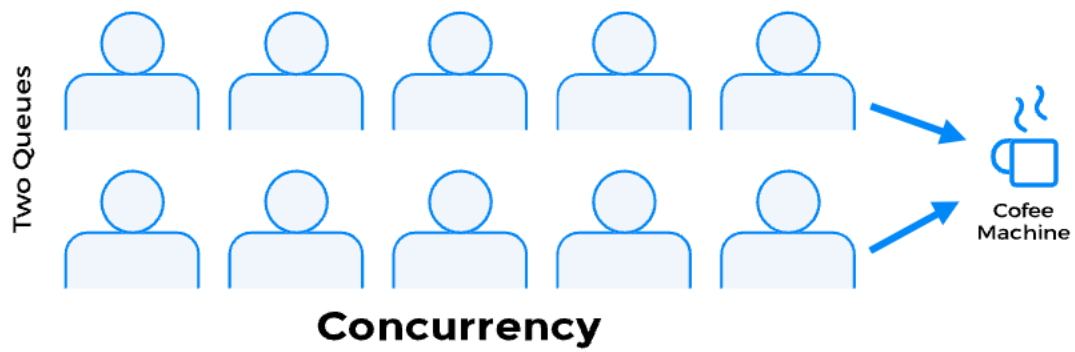
Parallelism refers to the ability of a system to execute multiple tasks or threads simultaneously, using multiple cores or processors.

Parallelism enables a system to perform multiple tasks at the same time, rather than one at a time.

For example, a parallel program can use multiple cores or processors to perform a large computation in less time.

In summary, **concurrency enables a program to handle multiple tasks at the same time**, but it does not necessarily mean that the tasks are executed simultaneously.

Parallelism enables a program to execute multiple tasks simultaneously, using multiple cores or processors.

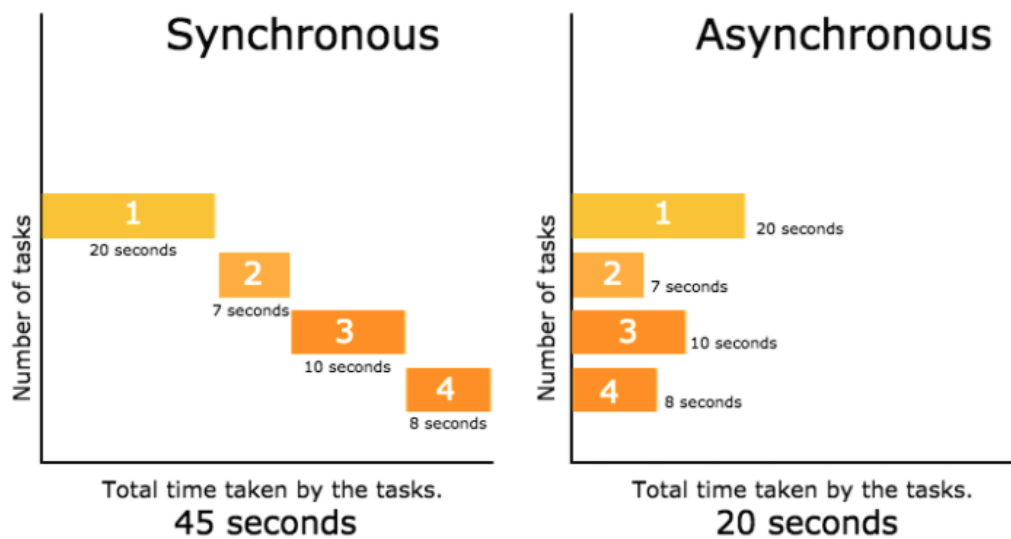


Synchronous vs Asynchronous

Synchronous operations block the execution of the program or system until the operation is completed.

This means that the program or system will not execute any other code or perform any other tasks until the current task is completed.

This can lead to a situation where the program or system is "stuck" waiting for an operation to complete, and this can make the program or system unresponsive or slow.

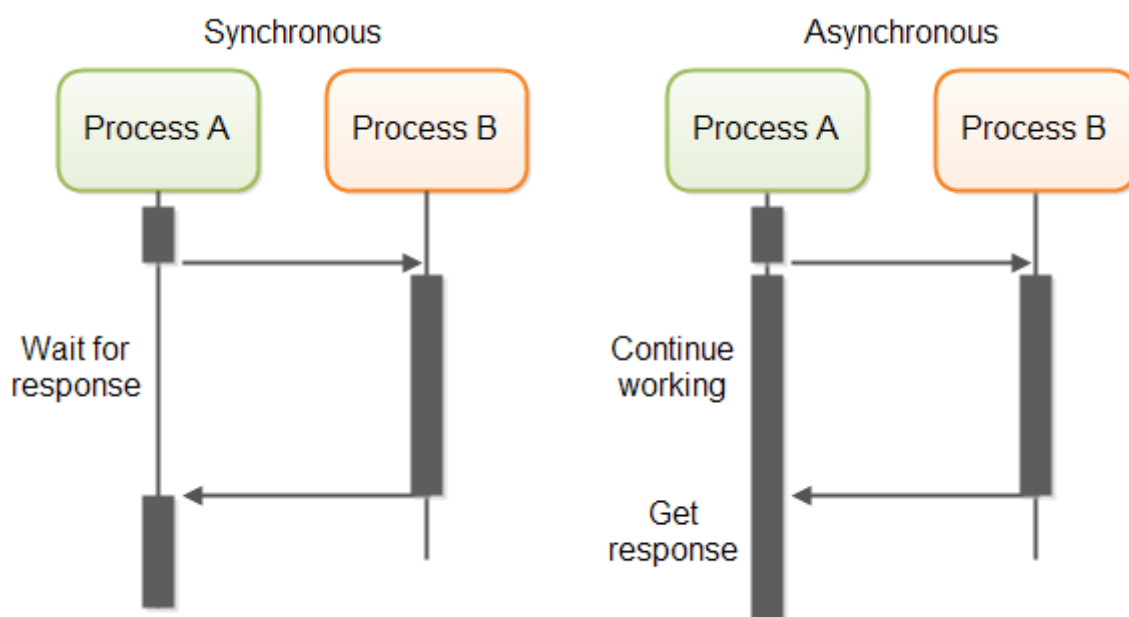


Asynchronous operations, on the other hand, do not block the execution of the program or system.

The program or system can continue to execute other code or perform other tasks while an asynchronous operation is in progress.

This can make the program or system more responsive and efficient, because it can perform multiple tasks at the same time.

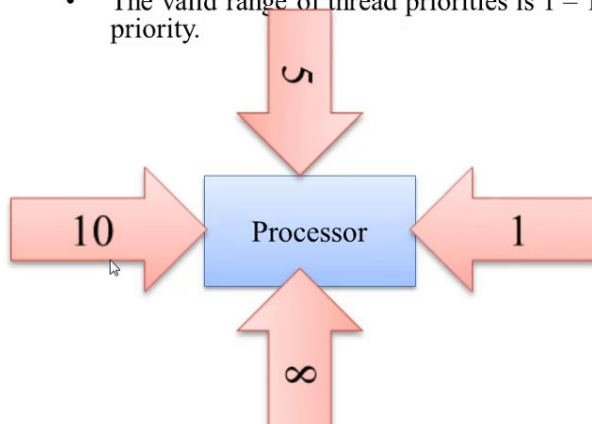
In summary, **synchronous operations block the execution of a program or system until the operation is completed, while asynchronous operations do not block the execution and allow the program or system to continue running while the operation is in progress.**



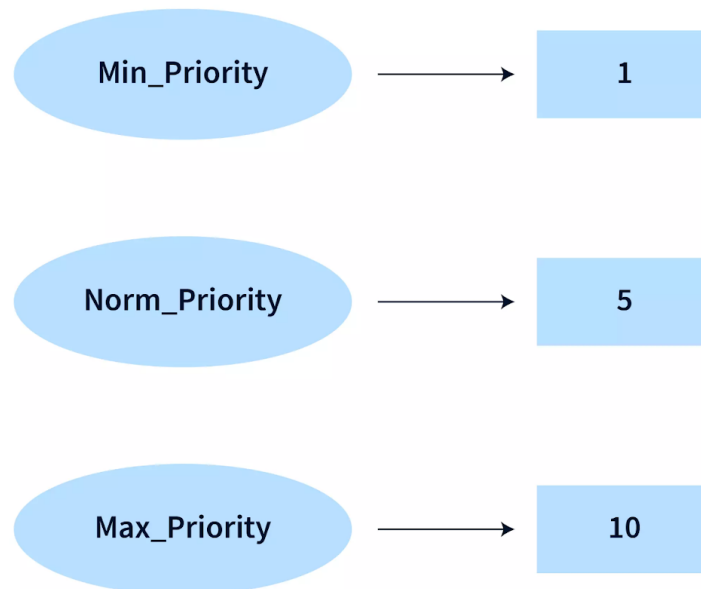
Thread Priority

Thread priority determines how a thread should be treated with respect to others.

- Every thread in java has some priority it may be default priority generated by JVM or customized priority provided by programmer.
- The valid range of thread priorities is 1 – 10, where 1 is min priority and 10 is max priority.



- Thread scheduler will use priorities while allocating processor.
- The thread which is having highest priority will get the chance first.



Java Thread Priority

You can create your own Thread Group also

Constants from Thread.

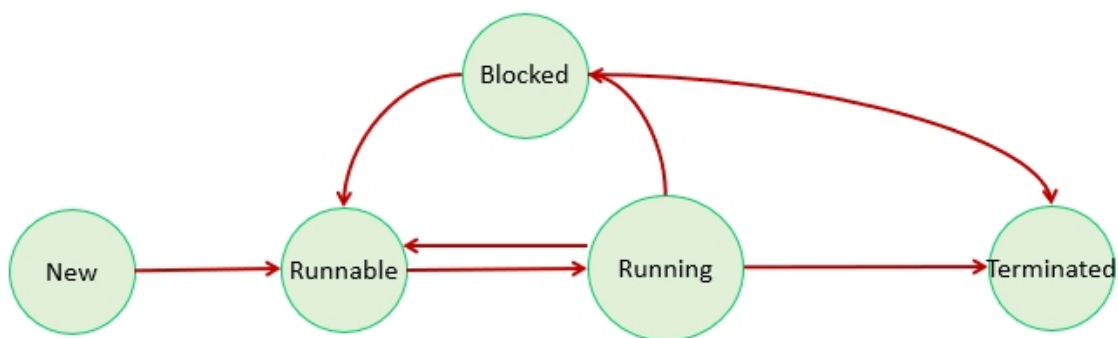
State enum

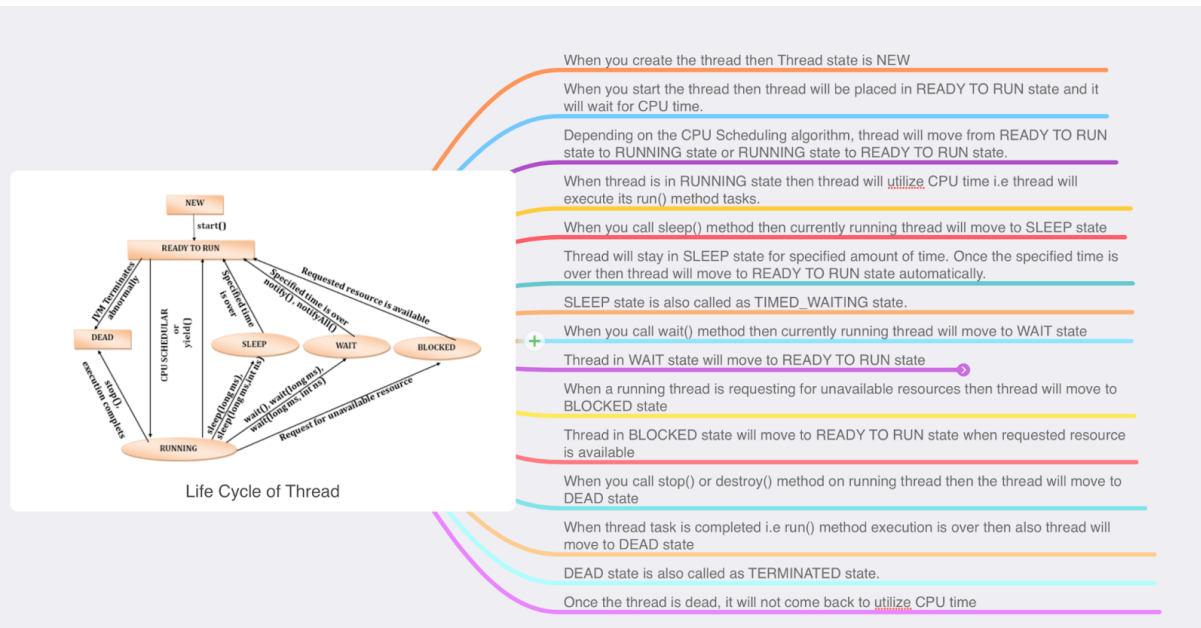
NEW RUNNABLE BLOCKED

WAITING TIMED_WAITING TERMINATED

Thread Life CYCLE

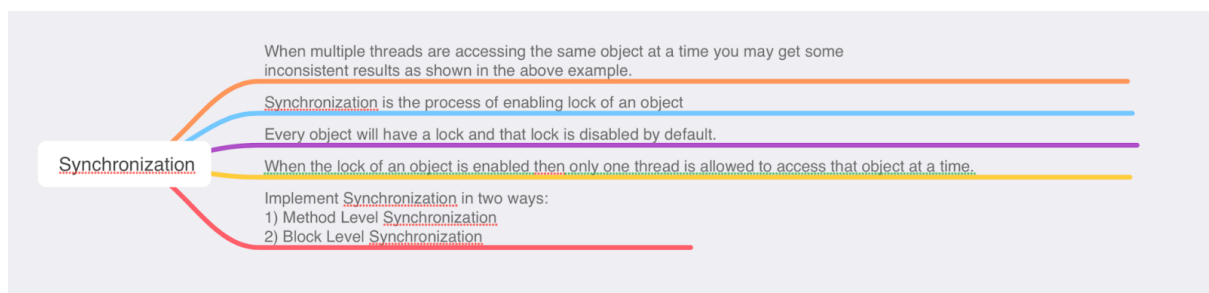
1. Create a Thread.
 2. Start the Thread (`th.start()`) , T1, T2.
 3. Initial state will be ready TO Run (waiting for CPU).
 4. Thread scheduler selects the t1
- New - Instance of thread created which is not yet started by invoking `start()` it will be.
 - Runnable - After Invocation of `start()` and before it is selected to be run by the scheduler.
 - Running - After the thread scheduler has selected it.
 - Non-runnable - Thread alive, not eligible to run.
 - Terminated - `run()` method has exited.





Synchronization

- Lock on an Object.



- Withdraw example for the Synchronization

Method Level Synchronization

- Applying synchronized keyword to the method is called Method Level Synchronization.
- You can apply synchronized keyword for both instance methods and static methods.
- When instance method is synchronized then the object which is used to invoke the synchronized method will be locked by the current thread.
- When static method is synchronized then the default object of the invoking class will be locked by the current thread.

```
public synchronized void show(){ ... }
```

Block Level Synchronization

- Applying synchronized keywords for local blocks is called as Block Level Synchronization.
- You must specify some object as a parameter to synchronized block.
- When a local block is synchronized, then the object which you are passing as a parameter to the synchronized block will be locked by the current thread.

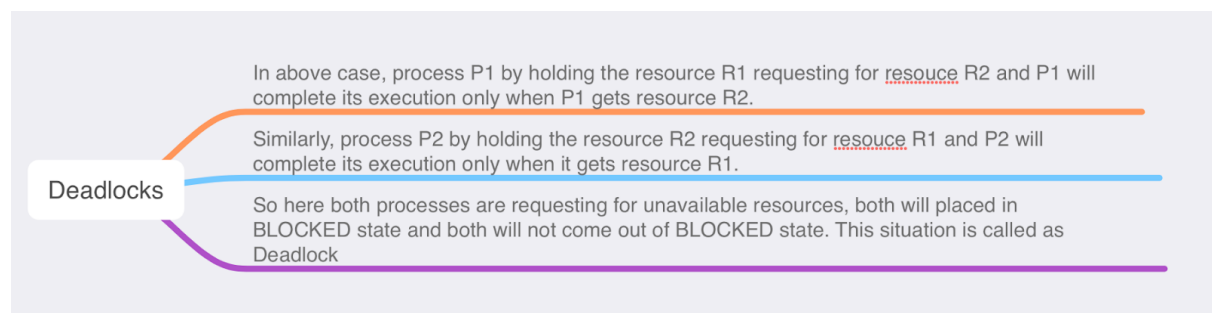
```
class Pramod{

ArrayList al = new ArrayList();
void show(){ ...
synchronized(al){ ... } ...
}
}
```

Important points for the Synchronization

- Case 1
- Case 2
- Case 3
- Case 4
- Case 5
- Case 6

Deadlocks



Solution in ATBNotes.zip (or Github URL)

```

package thetestingacademy.multithreading.deadlock;

public class ThreadDeadLock {

    public static void main(String[] args) {
        final String resource1 = "ATB";
        final String resource2 = "MTB";
        // t1 tries to lock resource1 then resource2
        Thread t1 = new Thread() {
            public void run() {
                synchronized (resource1) {
                    System.out.println("Thread 1: locked resource 1");

                    try {
                        Thread.sleep(100);
                    } catch (Exception e) {
                    }

                    synchronized (resource2) {
                        System.out.println("Thread 1: locked resource
2");
                    }
                }
            }
        };

        // t2 tries to lock resource2 then resource1
        Thread t2 = new Thread() {
            public void run() {
                synchronized (resource2) {
                    System.out.println("Thread 2: locked resource 2");

                    try {
                        Thread.sleep(100);
                    } catch (Exception e) {
                    }

                    synchronized (resource1) {
                        System.out.println("Thread 2: locked resource
1");
                    }
                }
            }
        };
    }
}

```

```

        t1.start();
        t2.start();
    }
}

```

Wait.

- wait() method is an instance method available in java.lang.Object class.
- When you call wait() method on any locked object then lock of that object will be released
- immediately and the current thread will be moved to WAIT state.
- Following are the wait() methods defined in Object class:
 - o public final void wait()
 - o public final native void wait(long ms)
 - o public final void wait(long ms, int ns)
- If you call wait() method on unlocked object then java.lang.IllegalMonitorStateException
- will be thrown at runtime

```

package thetestingacademy.multithreading.synchronization.WaitDemo;

public class WaitDemo {

    public static void main(String[] args) {

        WaitForMe waitForMe = new WaitForMe();

        CustomThread customThread = new CustomThread(waitForMe, "A");
        CustomThread customThread2 = new CustomThread(waitForMe, "B");

        customThread.start();
        customThread2.start();
    }
}

class WaitForMe {

    //    synchronized void show(){
    synchronized void show() {
        Thread thread = Thread.currentThread();
        for (int i = 0; i < 5; i++) {
            System.out.println(" Name -> " + thread.getName() + " -> " +
i);
            try {
                this.wait(1000);

```

```

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

// Blocking a Block or Third Party Object
// void show() {
//     Thread th = Thread.currentThread();
//     ArrayList al = new ArrayList();
//     synchronized (al) {
//         for (int i = 1; i <= 5; i++) {
//             System.out.println(th.getName() + " - show -" + i);
//             try {
//                 this.wait(1000);
//                 al.wait(1000);
//             } catch (Exception ex) {
//                 System.out.println(ex);
//             }
//         }
//     }
// }
}

```

```

class CustomThread extends Thread {
    WaitForMe wait;

    public CustomThread(WaitForMe wait, String name) {
        super(name);
        this.wait = wait;
    }

    @Override
    public void run() {
        wait.show();
    }
}

```

Notify & NotifyAll()

- These methods are instance methods available in java.lang.Object class.
- When you call notify() method on any locked object then following task will happen:
 - Only one thread which has been waiting for long time will be picked.

- Checks whether the resources released by the thread are available or not.
 - If resources are available, then the thread acquires the resources and goes to READY TO RUN state.
 - If resources are not available then the thread goes to BLOCKED state.
- When you call notifyAll() method on any locked object then following task will happen:
 - All or some required number of threads as per the requirement which are in WAIT state will be picked.
 - Checks whether the resources released are available or not.
 - If resources are available then the threads acquire the resources and goes to READY TO RUN state.
 - If resources are not available then the threads go to BLOCKED state.

Code sample at ATBNotes.zip

Join()

- This method will be used to join one thread at the end of another thread.
- In Main thread -> t1.join()
 - Main thread will be joined at the end of t1.
- In t1 thread -> t2.join()
 - t1 thread will be joined at the end of t2.

Thread Pool

- A thread pool in Java is a collection of worker threads that are used to execute multiple tasks concurrently.
- The main advantage of using a thread pool is that it allows the program to reuse threads, which can improve performance and reduce the overhead of creating and destroying threads.
- In a thread pool, a fixed number of worker threads are created and are managed by a thread pool executor. When a task is submitted to the thread pool, it is placed in a task queue and is executed by an available worker thread. Once a task is completed, the worker thread is returned to the thread pool and is available for executing another task.
- Java provides the Executor framework, which simplifies the task of creating and managing thread pools. The Executor interface has a single execute() method, which is used to submit tasks to the thread pool

```
Executor executor = Executors.newFixedThreadPool(5);
```

```
executor.execute(new RunnableTask());
```

Thread Local

ThreadLocal is a class in the Java standard library that allows you to create variables that are specific to a single thread. These variables are called "thread-local" variables, because they are local to the thread that accesses them.

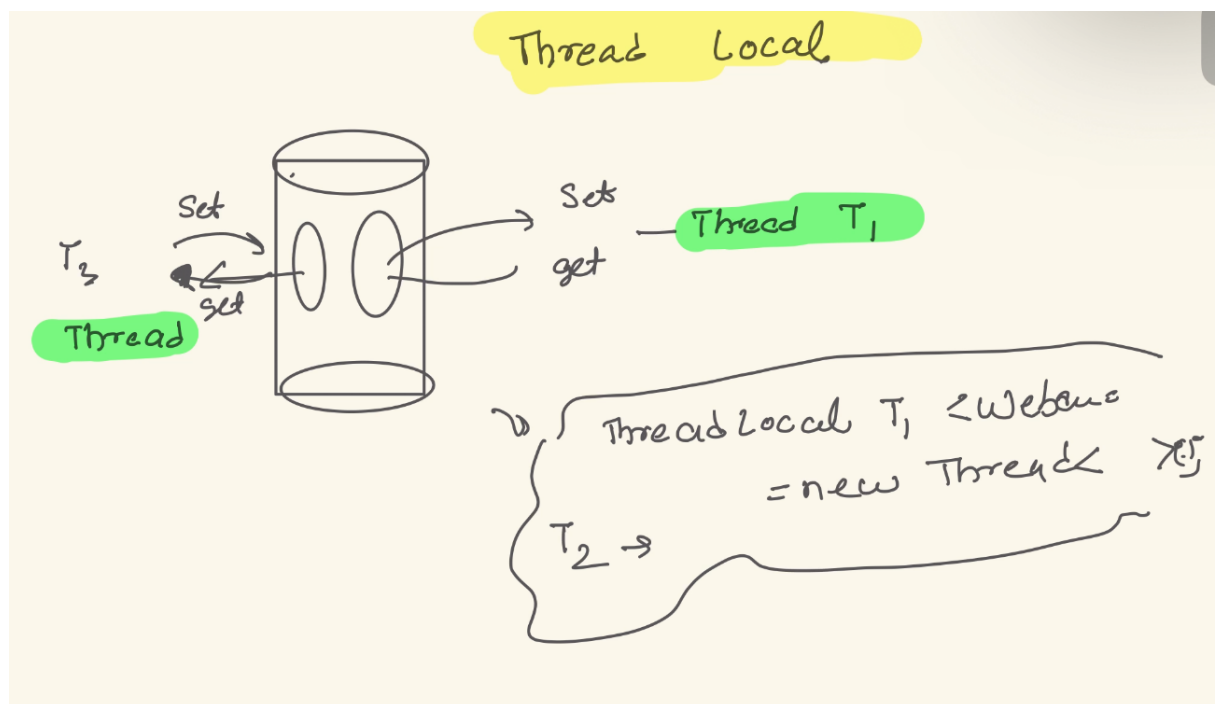
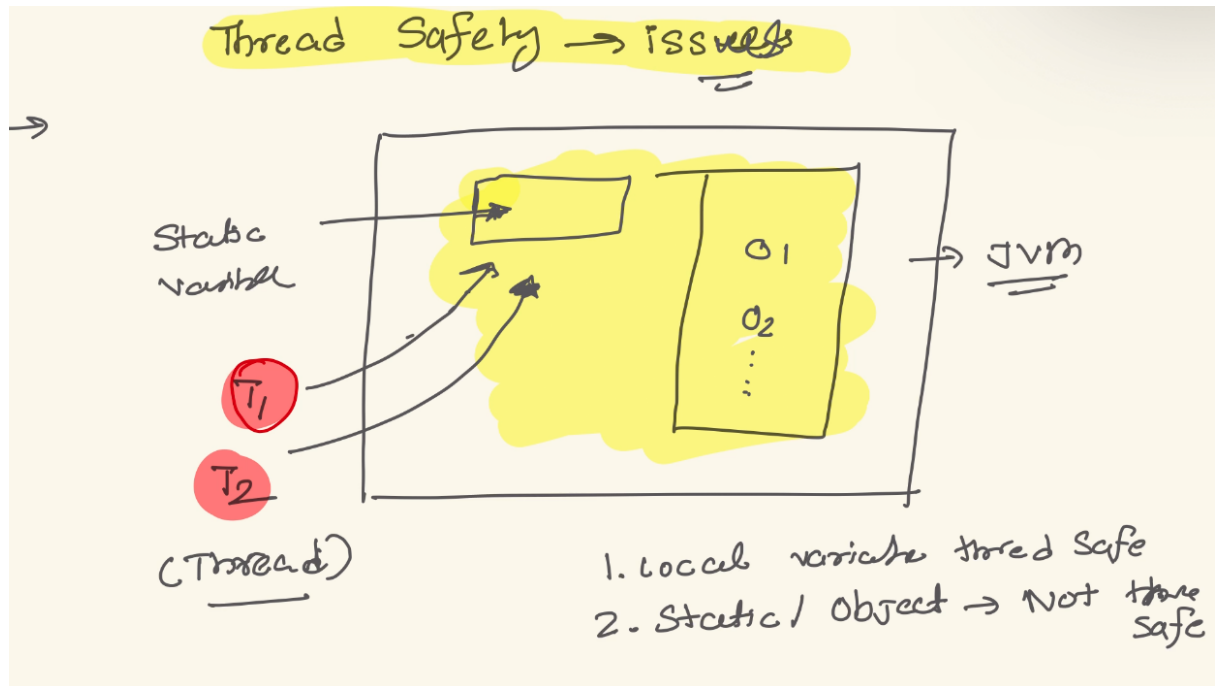
Instance variables are more prone to the Thread Safety, Local variables are always safe.

```
class MyThreadLocal {  
    private static ThreadLocal<Integer> myThreadLocal = new  
ThreadLocal<>();  
    public static void set(Integer value) {  
        myThreadLocal.set(value);  
    }  
    public static Integer get() {  
        return myThreadLocal.get();  
    }  
}
```

```
MyThreadLocal.set(10);  
System.out.println(MyThreadLocal.get()); // 10
```

It's important to note that each thread will have its own copy of the thread-local variable, so the value of the thread-local variable set by one thread will not affect the value of the thread-local variable in another thread.

For example, ThreadLocal can be used to store a user's session information in a web application, where each user's session information is specific to the thread that is handling the user's request.



What is the finalizer Thread?

- A finalizer thread is a special type of thread in Java that is used to run the finalize() method of an object before it is garbage collected.
- The finalize() method is a protected method that is defined in the Object class and can be overridden by subclasses to perform cleanup tasks, such as releasing resources or closing open file handles.

 Copy code

```
import java.io.File;

public class MyResource {
    private File file;
    // constructor and other methods

    protected void finalize() throws Throwable {
        try {
            file.delete();
        } finally {
            super.finalize();
        }
    }
}
```

In this example, the `finalize()` method is used to delete a file when the `MyResource` object is eligible for garbage collection.

However, it's important to note that the `finalize()` method is not guaranteed to be run before an object is garbage collected, and it's not guaranteed to be run at all.

So, it's not recommended to rely on `finalize()` method to release resources, instead use try-catch-finally or try-with-resources statements to release resources and also use explicit methods like `close()` to release resources.

it's not recommended to use `finalize()` method for critical tasks, because it's not guaranteed to be run and it might cause performance issues.

Can I overload the `run()` method in thread class?

Overloading of `run()` method is possible

Can I define the `run()` method as synchronized?

Yes, we can synchronize a `run()` method in Java, but it is not required because this method has been executed by a single thread only.

Can I define a thread class without overriding the `run ()` method?

If we don't override `run()` method, compiler will not flash any error and it will execute `run()` method of `Thread` class that has empty implemented, So, there will be no output for this

thread.

How do I pause thread execution for specified amount of time?

sleep() method can be used to pause the execution of the current thread for a specified time in milliseconds.

What is the difference between sleep() and wait() method?

Wait() method releases lock during Synchronization. Sleep() method does not release the lock on object during Synchronization.

What is daemon thread?

Daemon thread in Java is a low-priority thread that runs in the background to perform tasks such as garbage collection.

public final void **setDaemon**(boolean on)

What is race condition in Java?

A condition in which the critical section (a part of the program where shared memory is accessed) is concurrently executed by two or more threads.